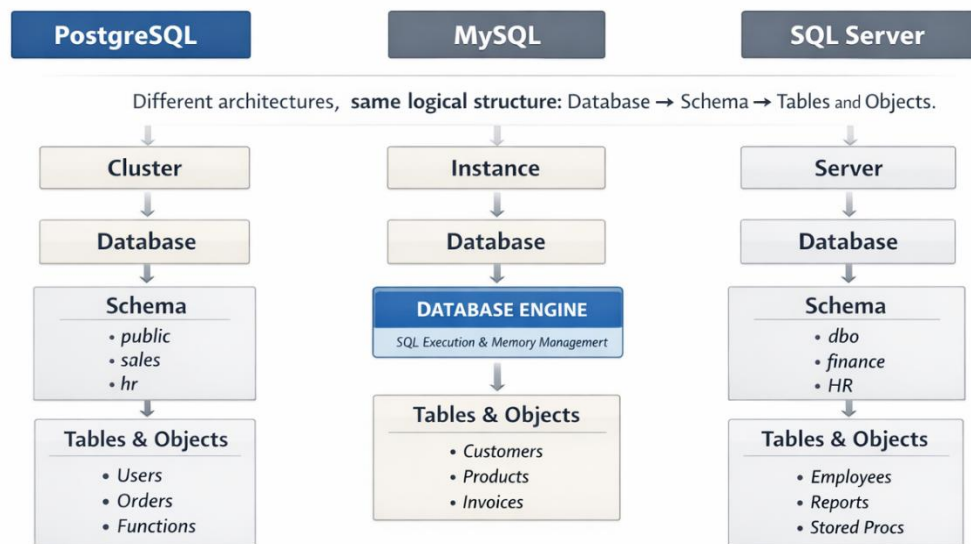


Relational Databases & Why We Use PostgreSQL

What a is Relational Database?

A relational database is a structured system that organizes data into tables connected by relationships, enabling efficient storage, retrieval, and manipulation through SQL. Although vendors like PostgreSQL, MySQL, SQL Server, and Oracle differ in implementation details such as how they handle storage engines, indexing, or transaction management. They all follow a consistent logical architecture.



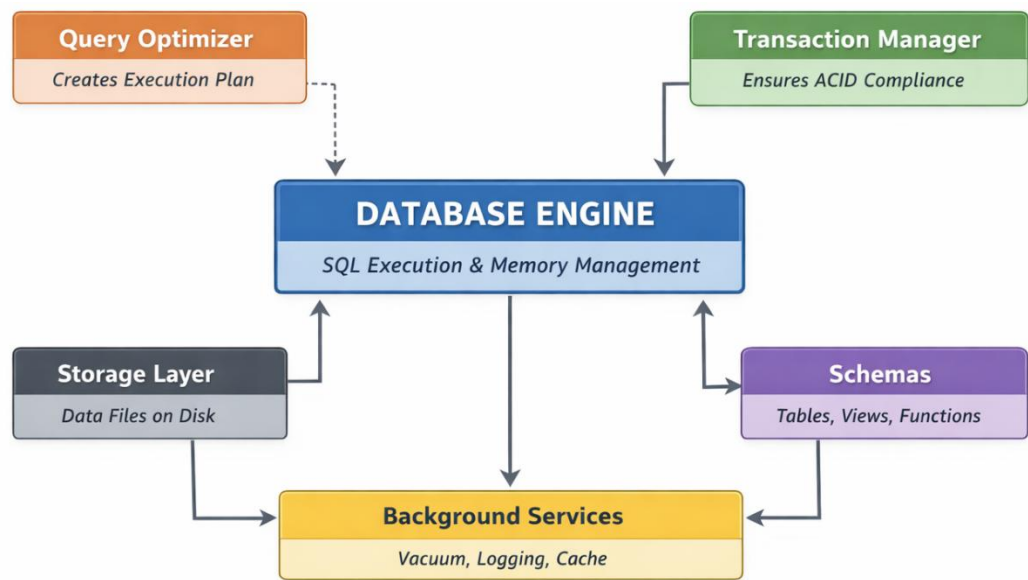
At its core, every relational database is built around a hierarchy that defines how data is organized and accessed: **Database → Schema → Database Objects (Tables, Views, Functions, Procedures, etc.)**.

- The **Database** acts as the top-level container, holding all data and metadata.
- Within it, **Schemas** serve as logical namespaces that group related objects, providing structure and access control.
- Inside each schema are **Database Objects**, which include the actual data structures (tables, views, indexes) and logic (functions, triggers, stored procedures).

This layered organization ensures consistency across vendors. For example, PostgreSQL uses clusters and schemas, MySQL simplifies the model by omitting schemas but still groups tables under databases, and SQL Server uses servers and schemas. Despite these variations, the conceptual flow remains identical: *data is always organized hierarchically from the database level down to individual objects*.

Understanding this common structure makes it easier to switch between systems and design portable and well-organized data models across different relational platforms.

Core Components of an RDBMS



Database Engine	Executes SQL, manages memory, and coordinates reads/writes
Storage Layer	physical files on disk where data is stored
Schemas	logical containers for tables, views, functions
Transaction Manager	Ensures atomicity, consistency, isolation, durability
Query Optimizer	decides the most efficient execution plan
Background Services	vacuuming, logging, checkpoints, caching

A **relational database** is built around a central component (**database engine**) and every other subsystem exists to support, inform, or extend what the engine does. The engine is responsible for interpreting SQL, managing memory, coordinating reads and writes, and ensuring that every operation follows the rules of the system.

To do this, it constantly interacts with several tightly connected components. It communicates with the **storage layer** to retrieve and persist data by pulling pages from disk into memory and writing changes back through a controlled and durable mechanisms.

It relies on **schemas** as the logical map of the database by using them to locate tables, views, and functions so it knows exactly where to route each query.

Every operation the engine performs is wrapped by the **transaction manager**, which enforces ACID guarantees coordinating commits, rollbacks, locks, and isolation levels to keep data consistent even when many users interact with the system at once.

Before executing any query, the engine hands it to the **query optimizer**, which evaluates possible execution paths and returns the most efficient plan; the engine then follows that plan step-by-step.

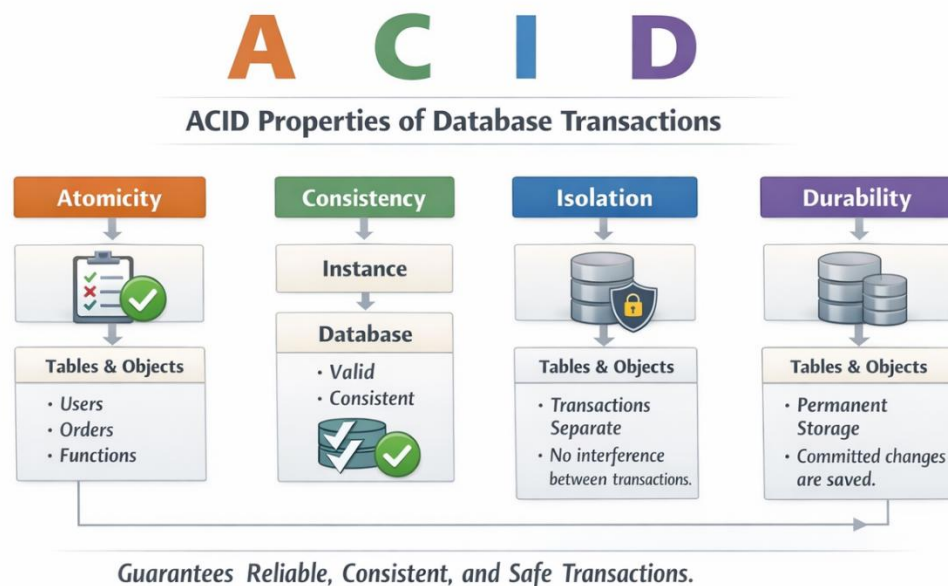
Surrounding all of this are **background services** that run continuously to maintain performance and data integrity. The engine orchestrates these services, ensuring they operate without disrupting active workloads.

Together, these components form a unified system where the database engine acts as the conductor that coordinates every part to deliver predictable and reliable relational behavior across vendors like PostgreSQL, MySQL, and SQL Server.

What is ACID?

The **ACID property** describes the four guarantees that every reliable relational database must uphold. These guarantees ensure that data remains correct, consistent, and durable even when multiple users, failures, or complex operations are involved. In practice, ACID is enforced by the **transaction manager** and coordinated by the **database engine**.

A transaction begins the moment a database operation starts, and ACID defines how that operation must behave from start to finish.



Atomicity means a transaction is treated as a single and indivisible unit of work; it either completes fully or has no effect at all. If any step fails, the entire transaction is rolled back, preventing partial updates.

Consistency ensures that every transaction moves the database from one valid state to another, respecting constraints, data types, relationships, and business rules. The database engine

validates these rules before committing changes and guaranteeing that no transaction can violate the defined structure of the data.

Isolation controls how concurrent transactions interact. Even when many users are reading and writing at the same time, each transaction behaves as if it is running alone. Databases achieve this through locking, MVCC, or snapshot isolation.

Durability guarantees that once a transaction is committed, its changes are permanent. Even if the system crashes immediately afterward, the data survives because the database has already written the changes to persistent storage.

Together, these four properties make relational databases predictable and trustworthy. Whether you're using PostgreSQL, MySQL, or SQL Server, ACID ensures that data remains accurate, consistent, and safe even under heavy load, hardware failures, or complex multi-step operations.

Differences between Postgres, MySQL and SQL Server

Feature	PostgreSQL	MySQL	SQL Server
SQL Compliance	<i>High</i>	<i>Medium</i>	<i>High</i>
Extensibility	<i>Excellent</i>	<i>Limited</i>	<i>Medium</i>
Licensing	<i>Free</i>	<i>Free (GPL)</i>	<i>Paid</i>
Best For	<i>Modern apps, analytics, APIs</i>	<i>Simple CRUD, legacy stacks</i>	<i>Enterprise systems</i>

A useful way to understand the difference between PostgreSQL, MySQL, and SQL Server is to look at the philosophy and ecosystem each one was built for.

PostgreSQL is an *object-relational* database designed for correctness, extensibility, and standards compliance. It supports advanced data types, custom functions, and a clean MVCC model. This makes it ideal for modern applications, analytics, and API-driven systems.

MySQL, on the other hand, was built for simplicity and speed, especially for web workloads. Its multiple storage engines, most notably InnoDB, allow developers to choose between durability and performance, which is why it became the backbone of countless PHP and LAMP-stack applications.

SQL Server is the enterprise-focused option, tightly integrated with the Windows ecosystem and supported by powerful tooling such as SSMS, SSIS, and SSRS. It excels in corporate environments that require strong administrative control, security, and reporting capabilities although licensing costs are a major consideration.

Despite these differences, all three systems share the same logical structure:

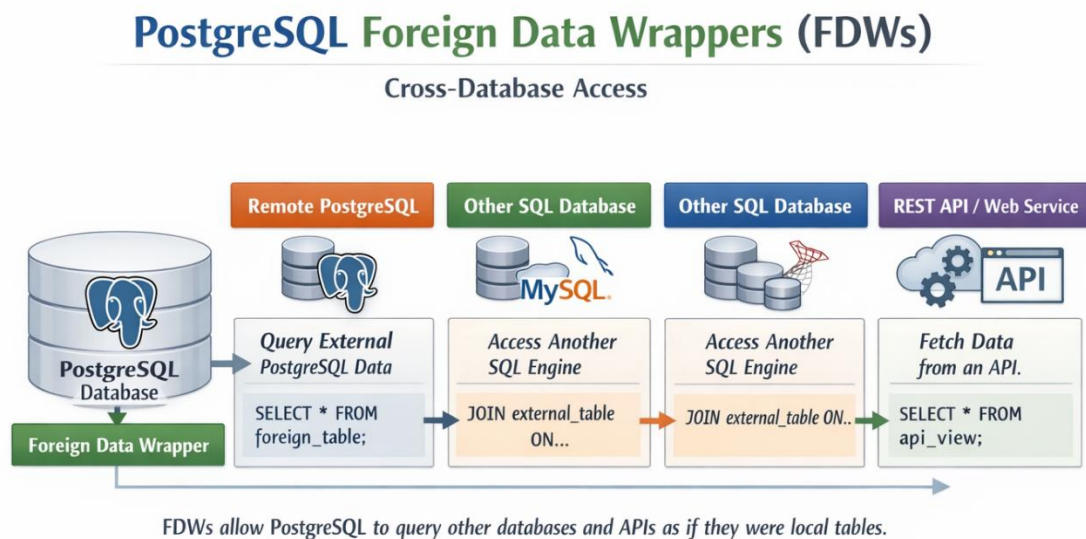
Database → Schema → Tables and Objects

Even if the terminology varies.

- PostgreSQL uses clusters and schemas,
- MySQL simplifies the model by omitting schemas
- SQL Server uses servers and schemas

but the conceptual hierarchy remains consistent.

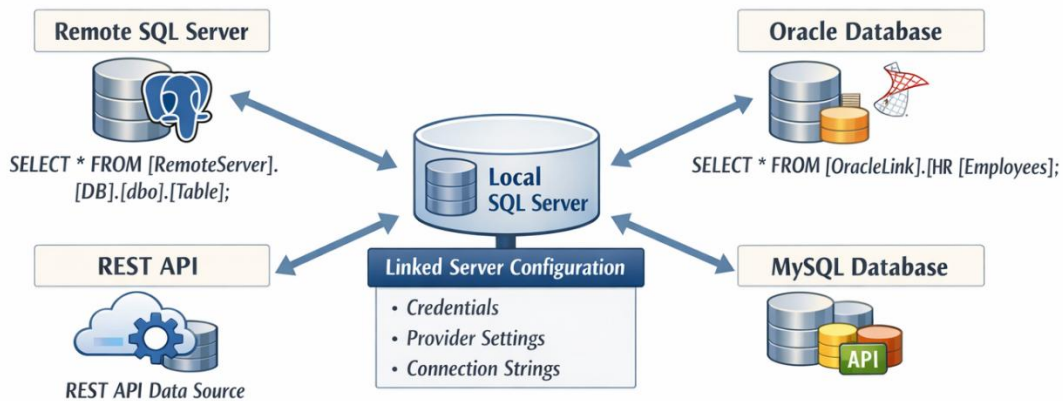
A key area where they differ significantly is cross-database access. PostgreSQL supports it through *Foreign Data Wrappers (FDWs)* which allows you to query external PostgreSQL databases, other SQL engines, or even APIs as if they were local tables.



SQL Server offers Linked Servers, enabling cross-server queries with T-SQL, often used in enterprise data integrations.

SQL Server Linked Servers

Cross-Server Queries with T-SQL

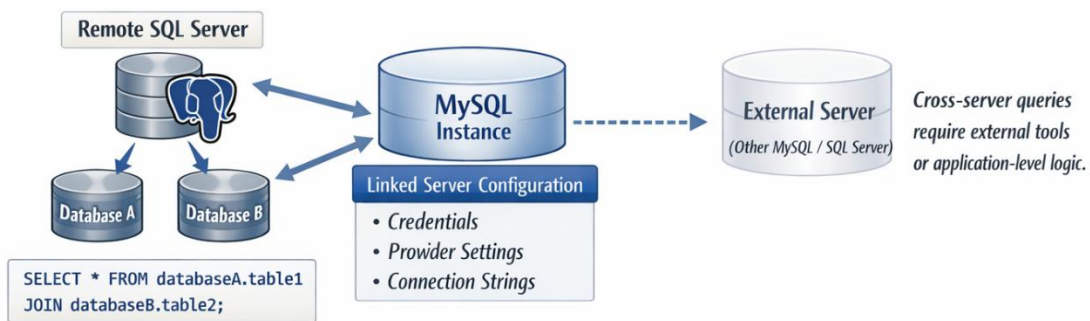


Linked Servers enable SQL Server to query external databases as if they were local — ideal for enterprise data integration.

MySQL, however, has limited native cross-database access; while you can query multiple databases within the same instance, true cross-server querying requires external tools or application-level logic.

MySQL Cross-Database Access

Cross-Server Queries with T-SQL



MySQL supports multi-database queries within one instance but lacks native cross-server access.

These differences reflect each system's design priorities: PostgreSQL emphasizes extensibility, SQL Server emphasizes enterprise integration, and MySQL emphasizes simplicity.

Understanding these distinctions helps engineers choose the right database for the right context:

- PostgreSQL for modern and data-rich systems
- MySQL for lightweight web applications
- SQL Server for large and structured corporate environments

Running PostgreSQL Locally (Installer Method)

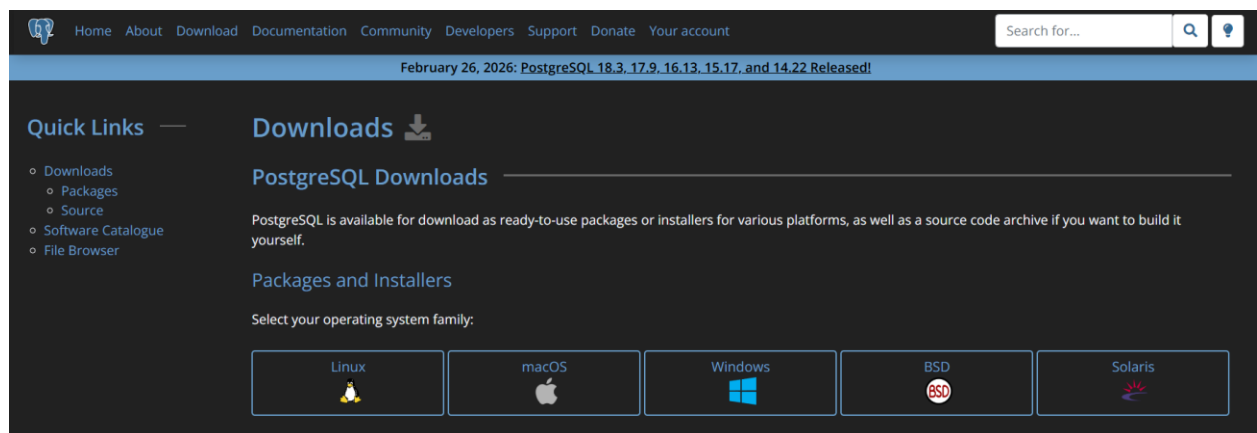
Why start with local installation

- No cloud costs: Everything runs on your machine, no billing surprises.
- No networking issues: No VPNs, firewalls, or security groups to debug.
- Faster feedback loop: Query, break things, and reset quickly.
- Beginner-friendly: Clear mental model “PostgreSQL is just another app on my computer.”

Step-by-step: Download to first connection

Step 1: Download the PostgreSQL installer

- Go to the official [PostgreSQL download page](#).



- Choose your OS (e.g., Windows) and download the latest stable version installer (often from EnterpriseDB).

Home About Download Documentation Community Developers Support Donate Your account Search for...

February 26, 2026: PostgreSQL 18.3, 17.9, 16.13, 15.17, and 14.22 Released!

Quick Links

- Downloads
 - Packages
 - Source
 - Software Catalogue
 - File Browser

Windows installers

Interactive installer by EDB

Download the [installer](#) certified by EDB for all supported Postgres versions.

Note! This installer is hosted by EDB and not on the PostgreSQL community servers. If you have issues with the website it's hosted on, please contact webmaster@enterprisedb.com.

This installer includes:

- The PostgreSQL server
- pgAdmin, a graphical tool for managing and developing your databases
- StackBuilder, a package manager for downloading and installing additional PostgreSQL tools and drivers. Stackbuilder includes management, integration, migration, replication, geospatial, connectors and other tools.

This installer can run in graphical or silent install modes.

The Windows installer is designed to be a straightforward, fast way to get up and running with Postgres. In the unlikely event that you have issues with the installer, please report them on the [installer Github page](#). Issues with the installed packages should be reported to the appropriate project directly.

Advanced users can also download a [zip archive](#) of the binaries, without the installer. This download is intended for users who wish to include Postgres as part of another application installer.

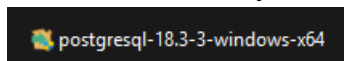
EDB POSTGRES AI Sign In Contact

Open source PostgreSQL packages and installers from EDB

PostgreSQL Version	Linux x86-64	Linux x86-32	Mac OS X	Windows x86-64	Windows x86-32
18.3	postgresql.org	postgresql.org			Not supported
17.9	postgresql.org	postgresql.org			Not supported
16.13	postgresql.org	postgresql.org			Not supported
15.17	postgresql.org	postgresql.org			Not supported
14.22	postgresql.org	postgresql.org			Not supported
13.23	postgresql.org	postgresql.org			Not supported

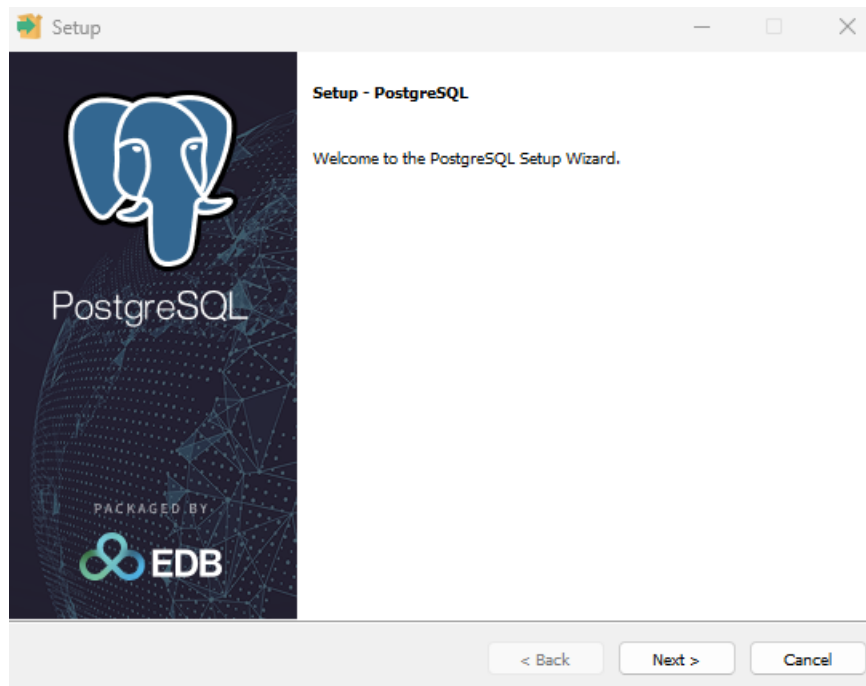
Note: EDB no longer provides Linux installers for PostgreSQL 11 and later versions, and users are encouraged to use the [platform-native packages](#). Version 10.x and below will be supported until their [end of life](#). For more information, please see this blog post on [Platform Native EDB Packages for Linux Users](#).

- Save the .exe file to your Downloads folder.

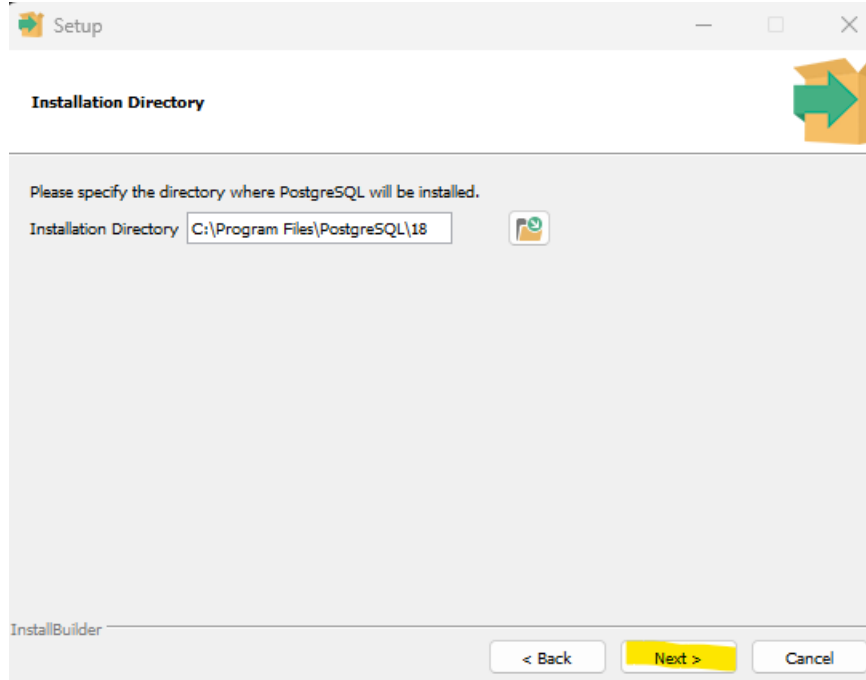


Step 2: Run the installer and select components

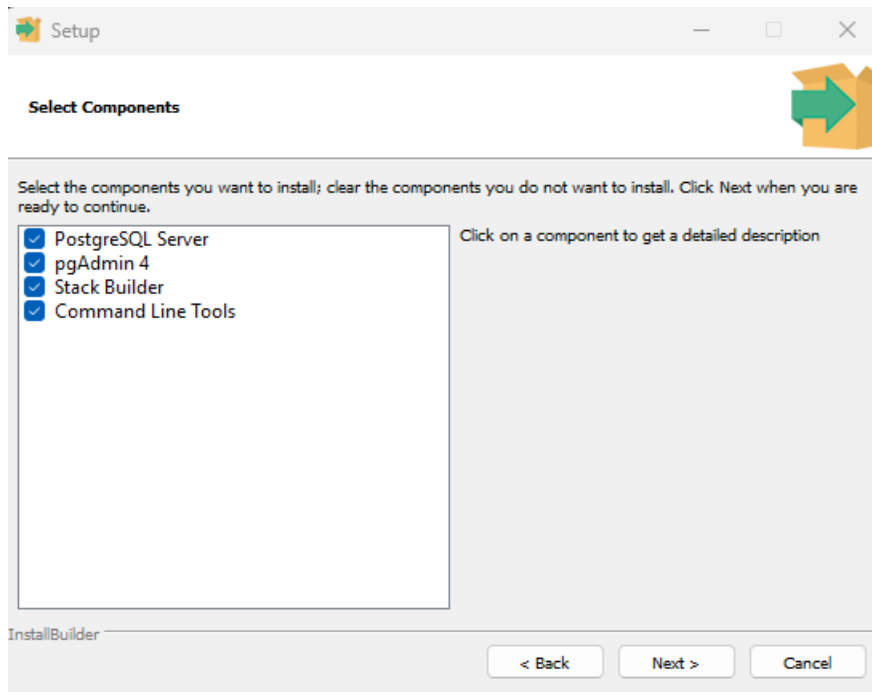
- Double-click the installer to start setup.



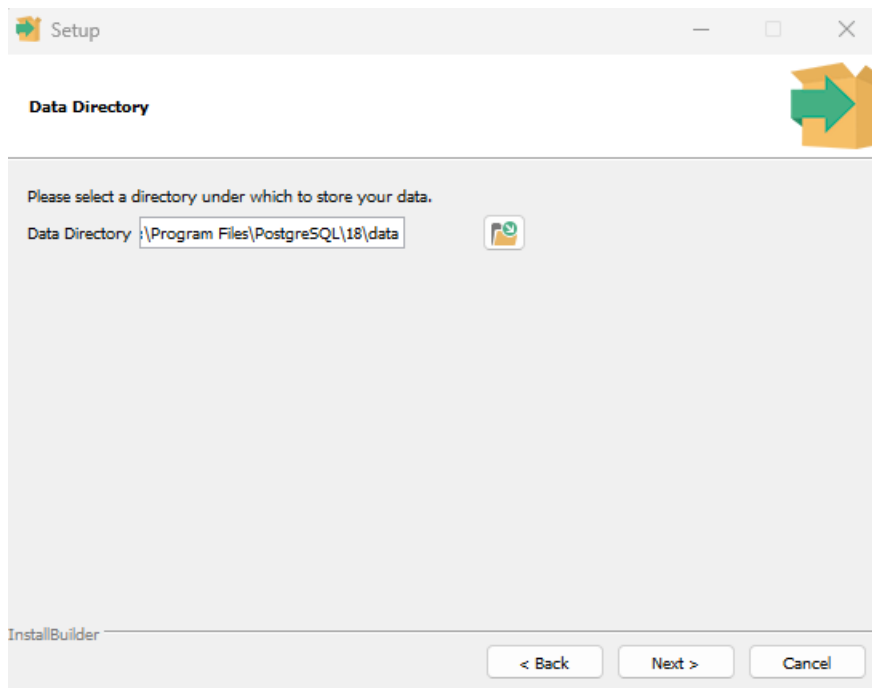
- Accept the default installation directory unless you have a reason to change it.



- When asked which components to install, keep the defaults:
 - PostgreSQL Server
 - pgAdmin (GUI)
 - Command Line Tools

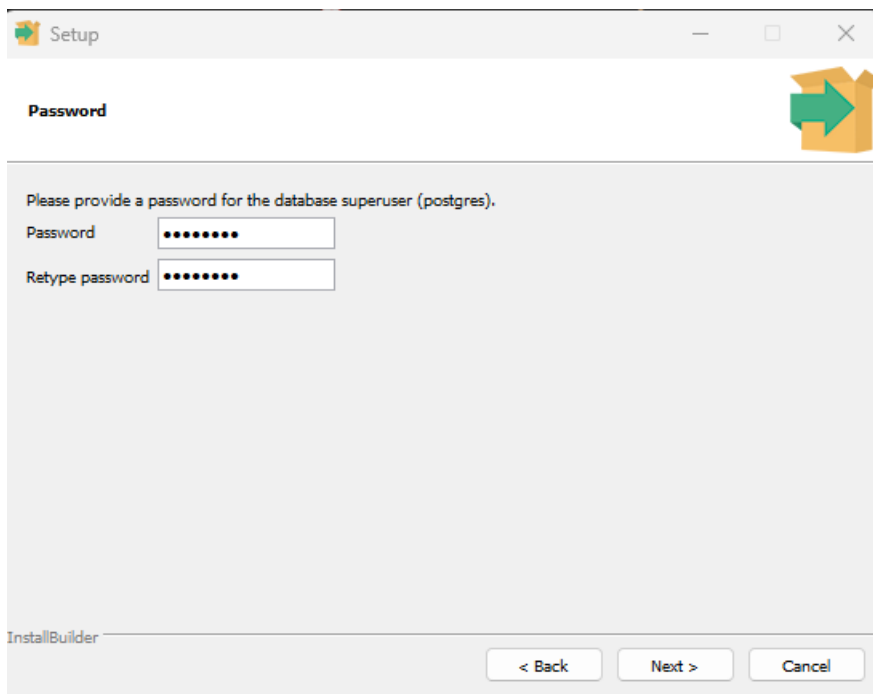


Click Next through the prompts.



Step 3: Set the PostgreSQL superuser password

- The installer will ask you to set a password for the default superuser account (usually *postgres*).



Setup

Password

Please provide a password for the database superuser (postgres).

Password

Retype password

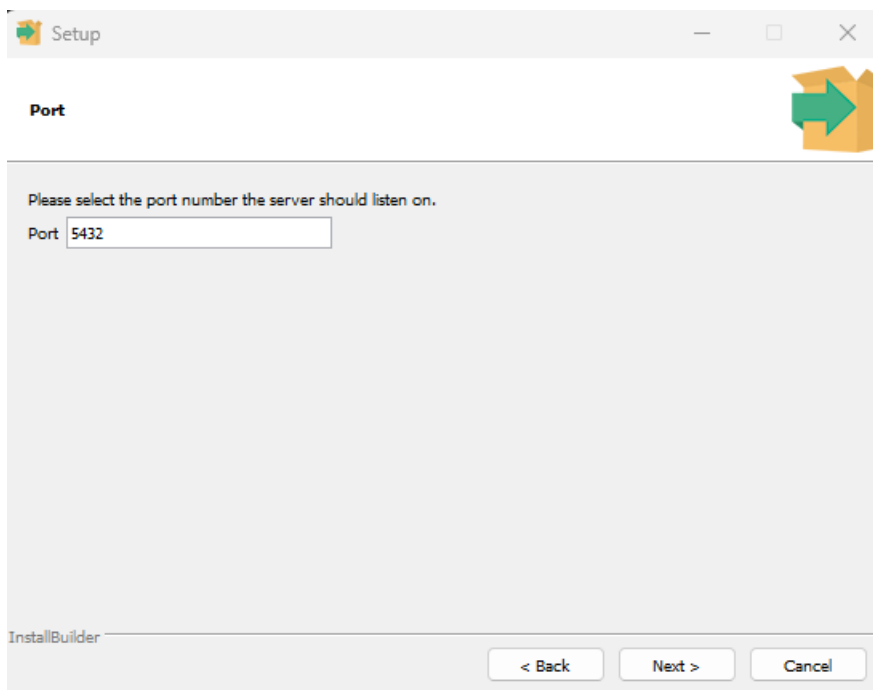
InstallBuilder

< Back Next > Cancel

- Choose a strong but memorable password.
- This password is required later when connecting via pgAdmin or any client.

Step 4: Configure port and service

- The default port is 5432 **leave it as is unless you have a conflict.**



Setup

Port

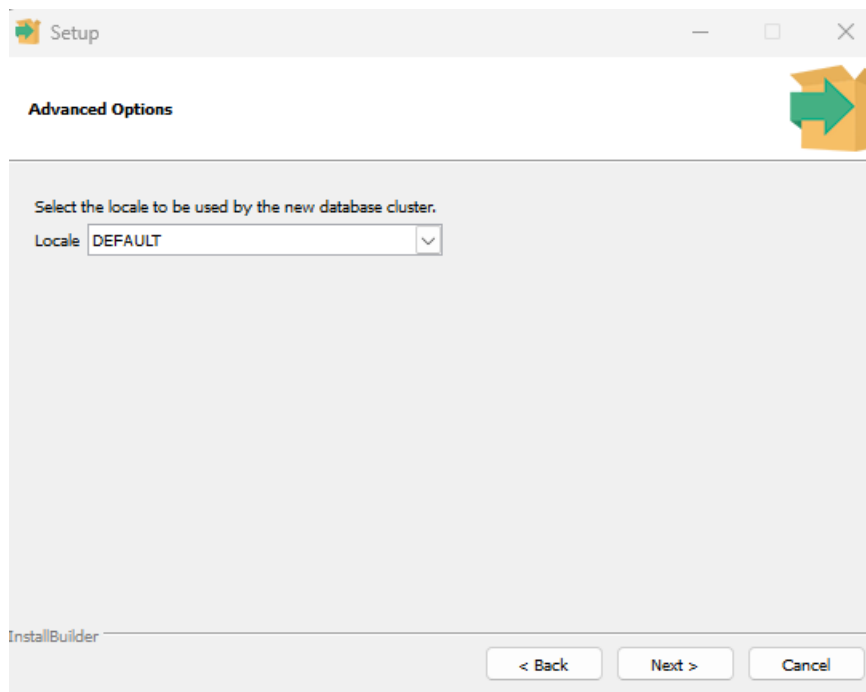
Please select the port number the server should listen on.

Port

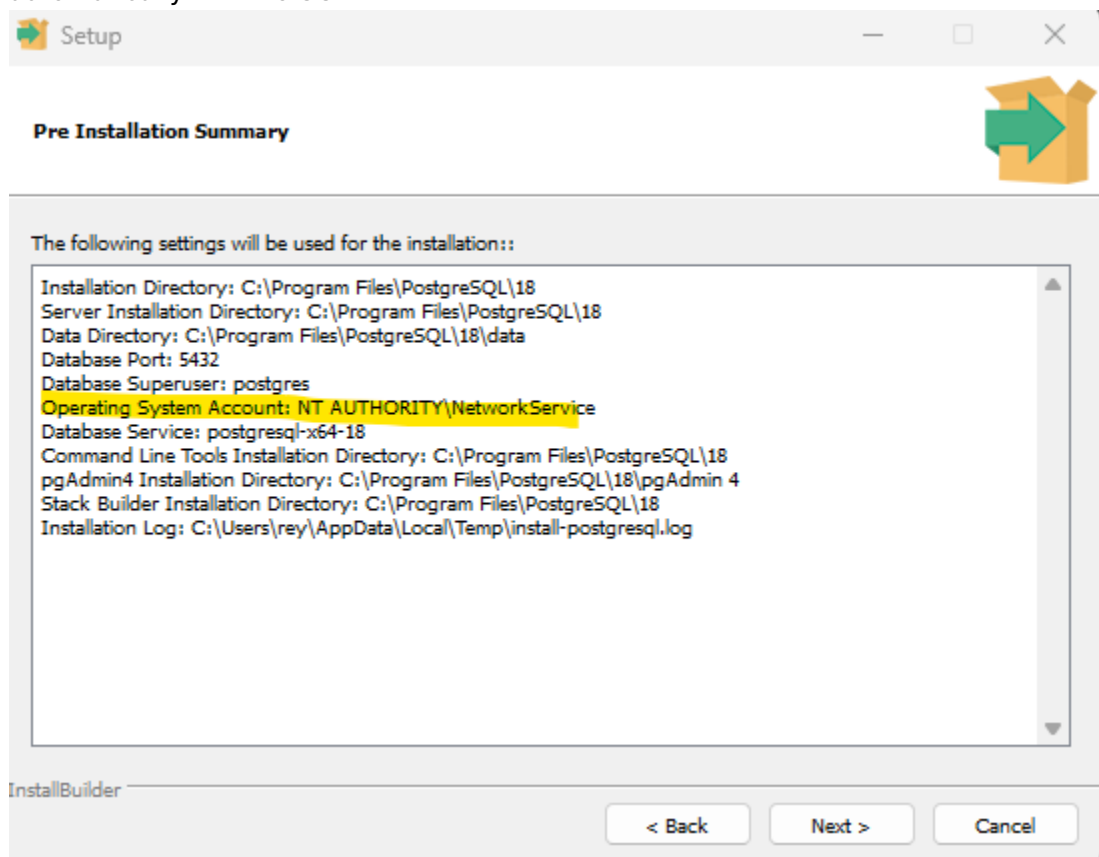
InstallBuilder

< Back Next > Cancel

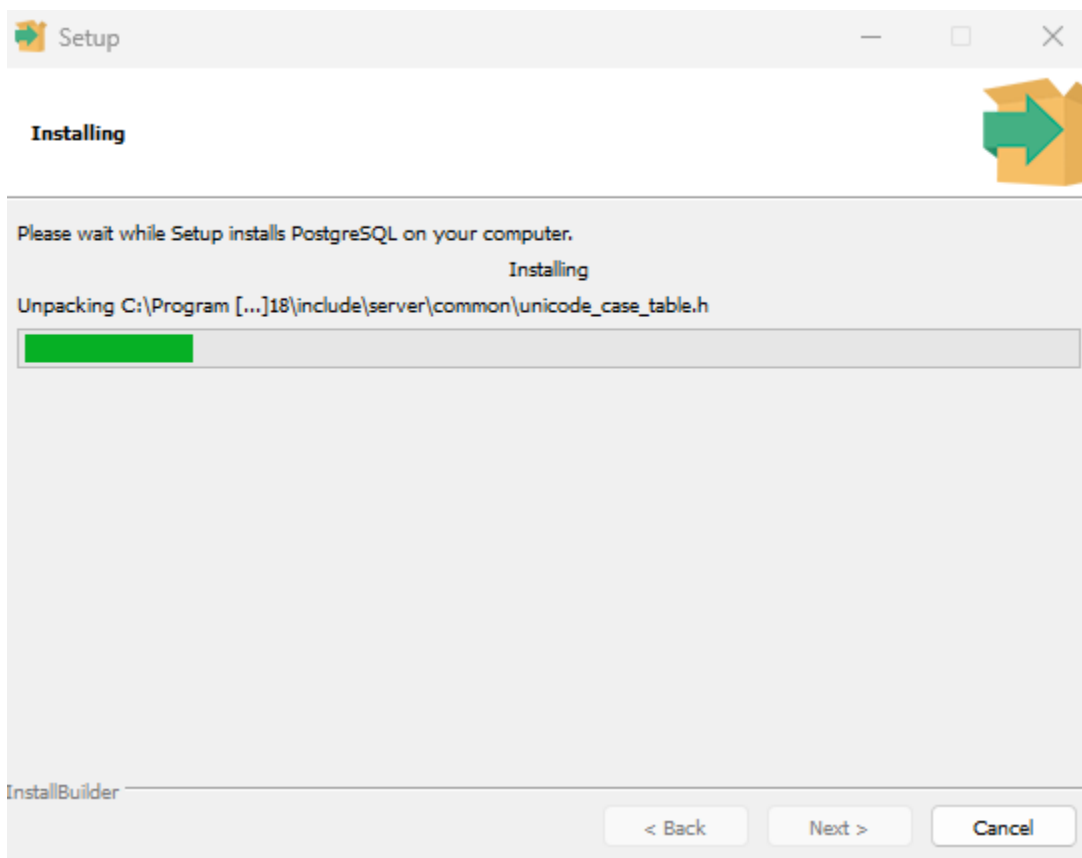
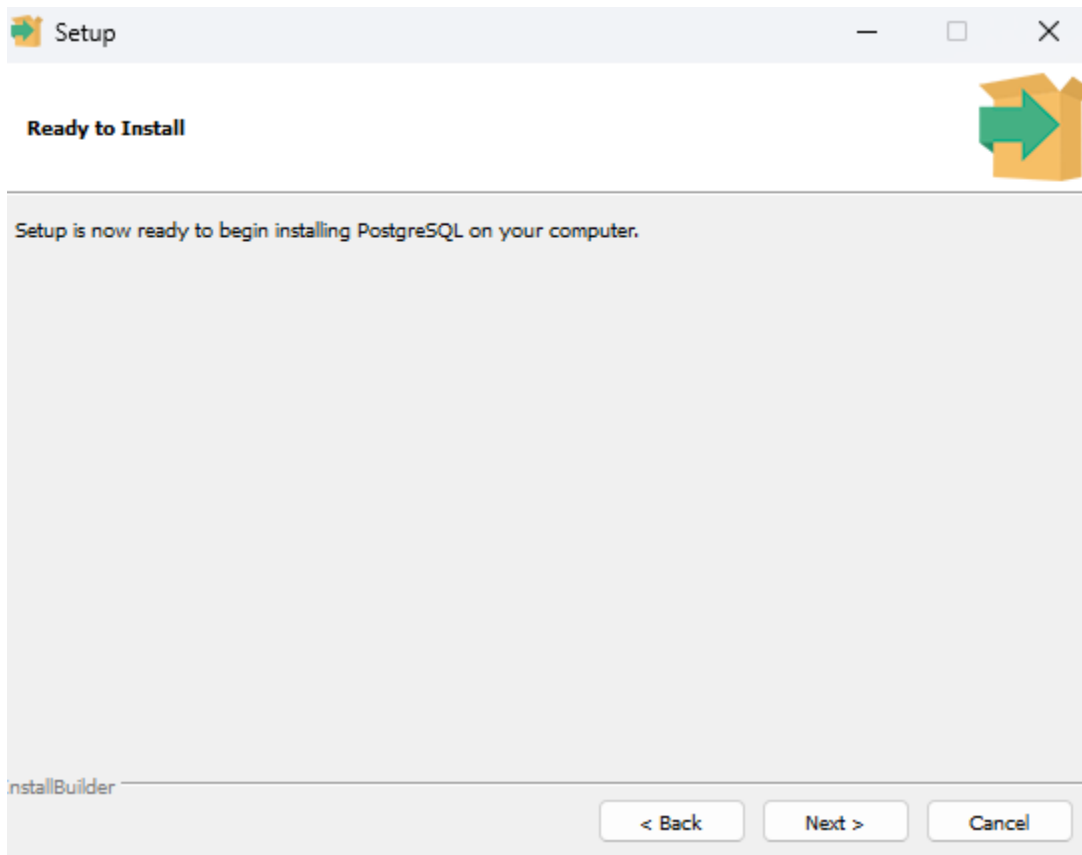
- Confirm the locale (usually fine to keep default).

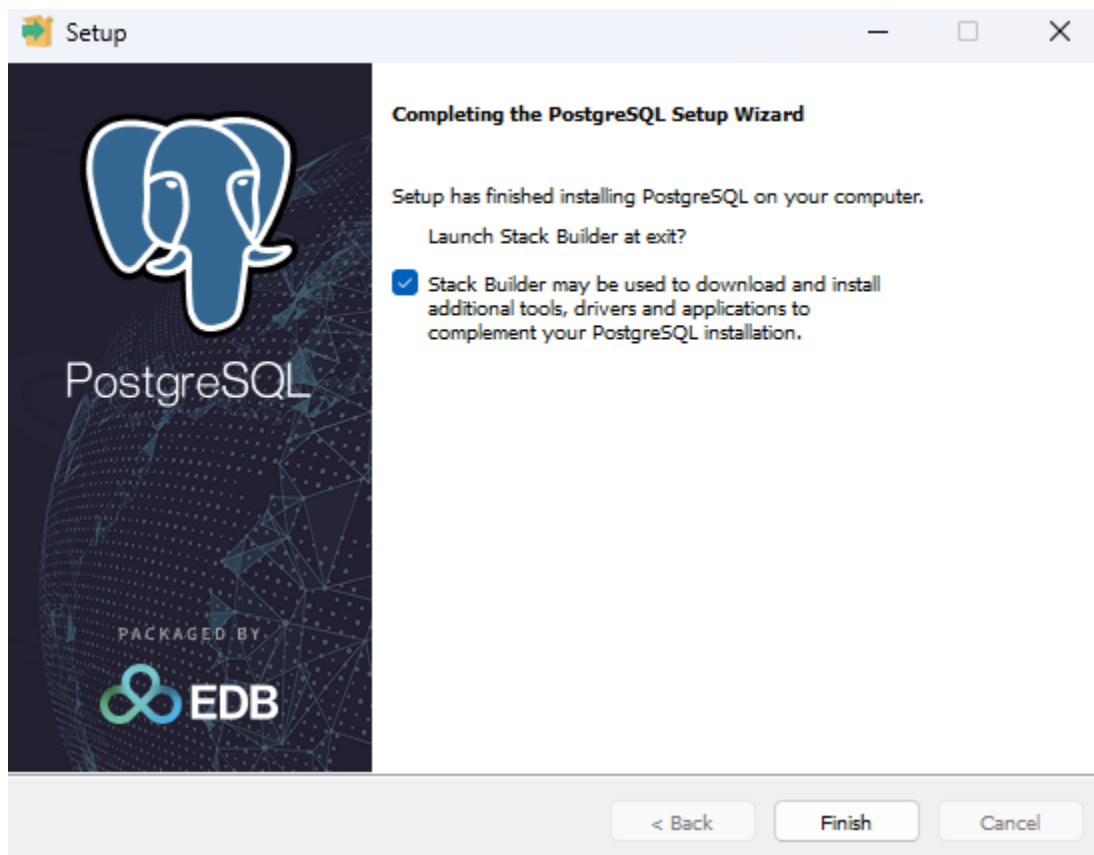


- The installer will register PostgreSQL as a Windows service, meaning it can start automatically with the OS.

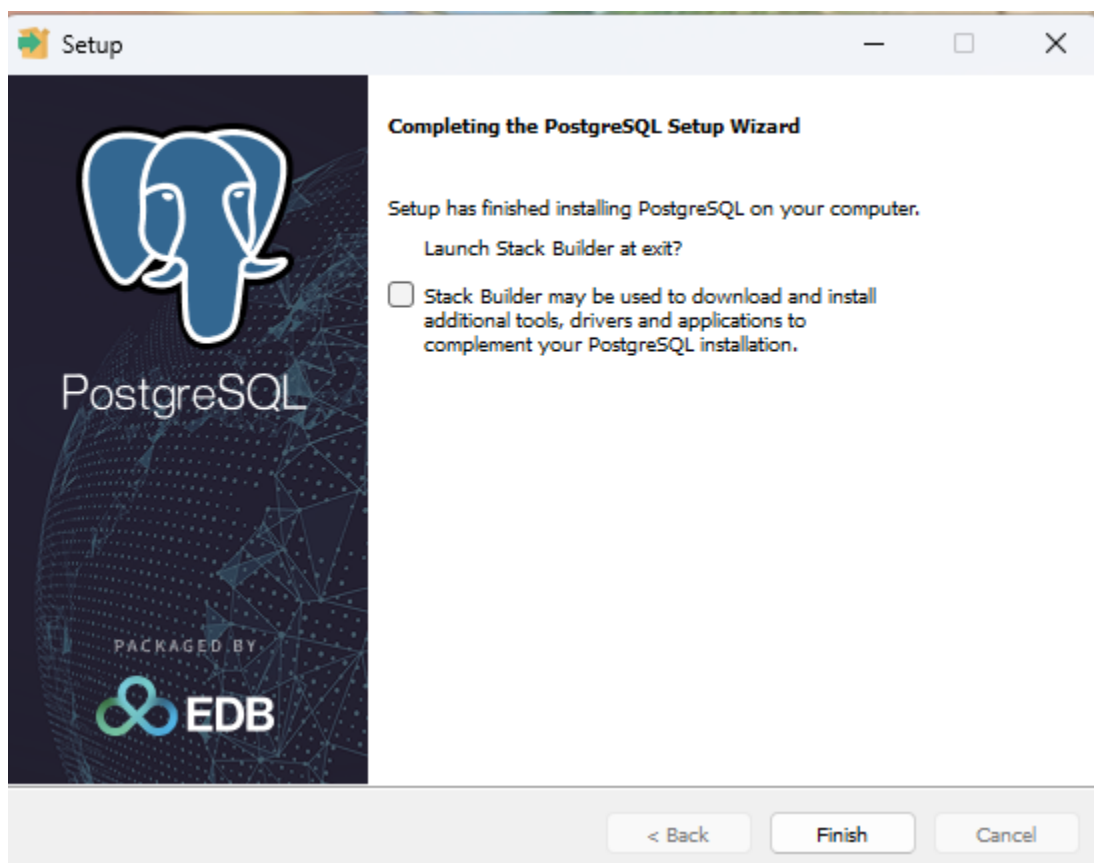


Click Next and then Finish to complete installation.



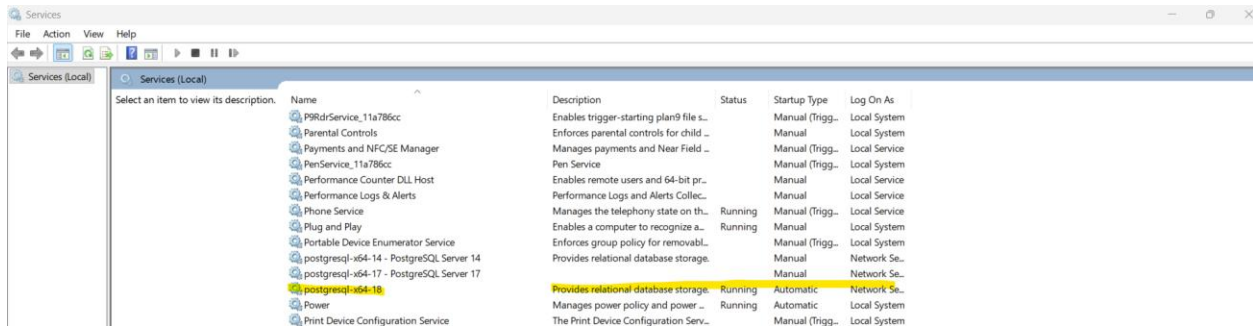


You can uncheck the Stack Builder. For this installation, we will uncheck it.

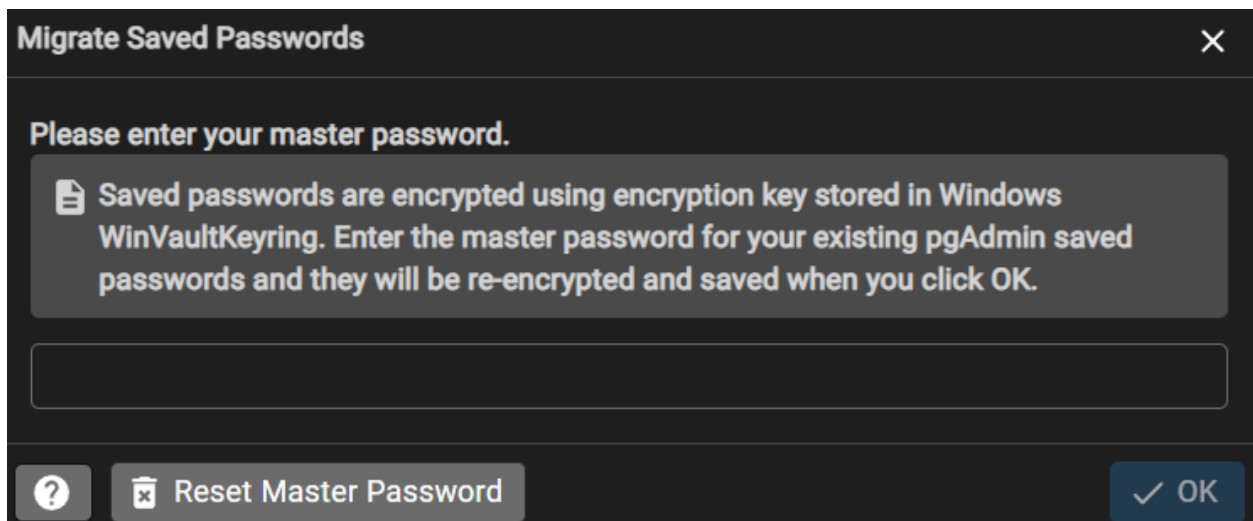


Step 5: Start PostgreSQL and open pgAdmin

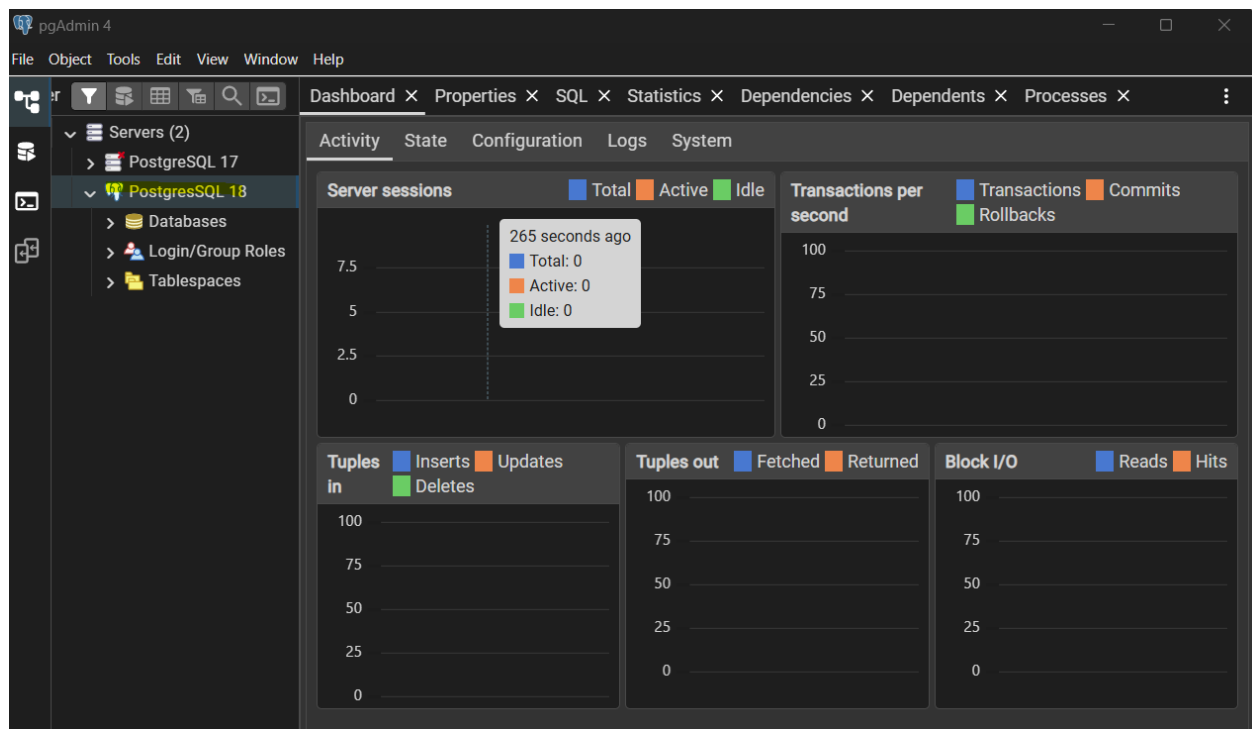
- After installation, PostgreSQL should already be running as a Windows service.



- Open pgAdmin from the Start Menu.
- The first time pgAdmin opens, it may ask you to set a master password (this is for pgAdmin, not PostgreSQL itself).
- In the pgAdmin tree, click on the PostgreSQL server entry.
- When prompted, enter the postgres password you set during installation.

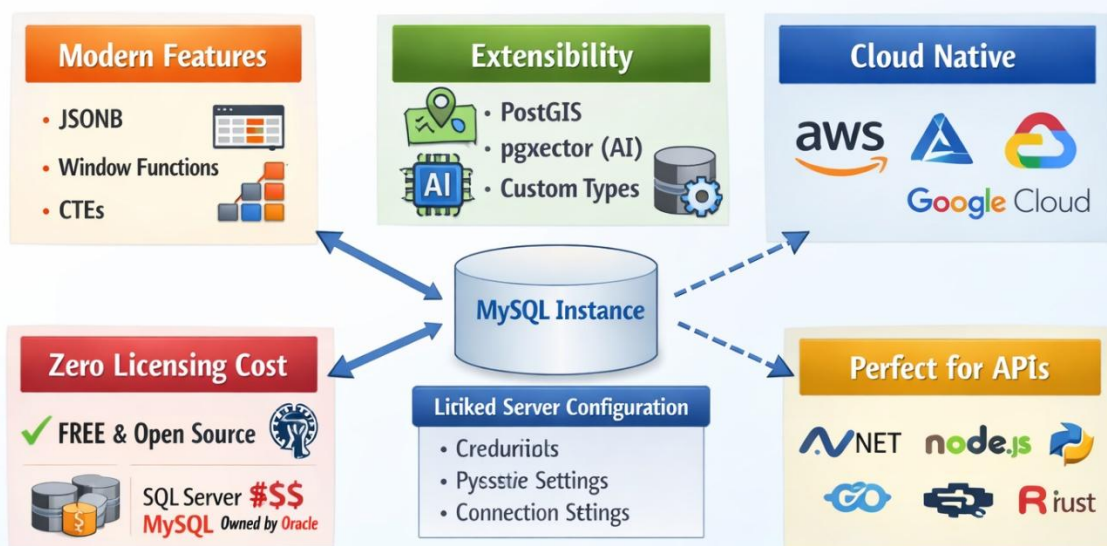


- Once connected, you can expand Databases, right-click, and create a new database for the class.



Why user Postgres?

Why We Use PostgreSQL in This Course



MySQL supports mullis-databass queries within one instance but lacks native cross server access.

We use PostgreSQL in this course because it represents the modern evolution of relational databases. It is flexible, standards-compliant, and deeply extensible. It's not just a database

engine; it's a platform for building data-driven systems. PostgreSQL supports advanced features like JSONB, window functions, CTEs, and extensions, which allow students to explore both traditional SQL and modern analytical patterns. These capabilities make it ideal for teaching how relational databases can handle semi-structured data and complex queries without sacrificing performance or clarity.

Its extensibility is what truly sets it apart. PostgreSQL can be customized and expanded through modules such as PostGIS for geospatial analysis and pgvector for AI embeddings, along with support for custom data types. This transforms PostgreSQL from a static data store into a dynamic platform that adapts to new domains (from mapping applications to machine learning pipelines).

PostgreSQL is also cloud-native and supported across major providers like AWS RDS, Azure PostgreSQL, and Google Cloud SQL. It ensures that students learn skills that translate directly to production environments. Unlike proprietary systems, it carries zero licensing cost. It is a major advantage for learners and startups alike. SQL Server's licensing can be expensive, and MySQL's governance under Oracle limits flexibility, while PostgreSQL remains community-driven and free.

Finally, PostgreSQL is perfect for APIs. It integrates cleanly with development ecosystems such as .NET, Node.js, Python, Go, and Rust, making it an ideal choice for full-stack projects. Students can see how SQL logic connects directly to application code, reinforcing the idea that PostgreSQL is not just a backend system but a foundation for modern software architecture.